

CREDIT CARD DEFAULT PREDICTION USING MACHINE LEARNING

PROJECT REPORT

Submitted as a part of the Summer Internship Program in Data Analytics
and Intelligence in Collaboration with Industry Partner AIHI Fusion
Technologies at JSS University Noida

Academic Year: 2026-27

Submitted by:

Member	Name	Roll No.	Branch	Institution
1.	Mayank Nigam	2400910130089	IT	JSSATEN
2.	Anshul Kanswal	J240211025	AIML	JSSUN
3.	Archi Jain	J240211007	AIML	JSSUN

GitHub Repository: <https://github.com/anshulkanswal6-dev/JSS-x-AIHI-Internship-Work>

Under the Guidance of

Dr. Prachi Chhabra & Dr. Sukhendra Singh
(Department of Information Technology)

CERTIFICATE

This is to certify that the internship project entitled “**Credit Card Default Prediction Using Machine Learning**” has been successfully completed by the following interns during the Summer Internship Program in Data Analytics and Intelligence at JSS University Noida:

Member 1: Mayank Nigam

Member 2: Anshul Kanswal

Member 3: Archi Jain

as a part of the **Data Analytics and Intelligence Summer Internship Program**.

The project was carried out under the guidance and supervision of **Dr. Prachi Chhabra & Dr. Sukhendra Singh**. During the internship, the team demonstrated dedication, analytical thinking, and technical proficiency in applying data analytics and machine learning techniques to develop a predictive model for identifying customers who are likely to default on their credit card payments.

The work presented in this report is original and has been completed during the internship period. To the best of our knowledge, this report has not been submitted elsewhere for the award of any degree, diploma, certificate, or internship requirement.

We wish the team success in their future academic and professional endeavors.

Project Mentor:

Signature: _____

Date: _____

DECLARATION

We hereby declare that the internship project report entitled "**Credit Card Default Prediction Using Machine Learning**" submitted as part of the **Data Analytics and Intelligence Summer Internship Program** is our original work carried out under the guidance of Dr. Prachi Chhabra & Dr. Sukhendra Singh

This project has been completed solely for academic and learning purposes during the internship. All information, analyses, results, and interpretations presented in this report are based on our own work, except where proper acknowledgment has been made through references.

We further declare that this report has not been submitted, either wholly or partially, to any institution, university, or organization for the award of any degree, diploma, certificate, or any other qualification.

Name	Signature
Member 1	_____
Member 2	_____
Member 3	_____

Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our **Project Mentor, Internship Coordinator**, and the entire **Data Analytics and Intelligence Internship Program** team for providing us with the opportunity to work on this project.

Their continuous guidance, valuable suggestions, encouragement, and technical support enabled us to understand the practical applications of machine learning in solving real-world financial problems. Their constructive feedback throughout the internship significantly enhanced our learning experience.

We would also like to thank all the faculty members, peers, and everyone who directly or indirectly contributed to the successful completion of this project.

Finally, we extend our heartfelt appreciation to our families and friends for their constant motivation, encouragement, and support throughout the internship.

ABSTRACT

Credit card default prediction is one of the most significant applications of machine learning in the financial sector, enabling banks and financial institutions to identify customers who are likely to default on future payments. Accurate prediction of default risk helps organizations reduce financial losses, improve credit risk management, and make informed lending decisions.

This internship project focuses on developing a machine learning model capable of predicting whether a credit card customer will default on the next month's payment based on demographic information, credit history, billing amounts, payment history, and other financial attributes. The project utilizes the **UCI Credit Card Default Dataset**, which contains customer information and repayment behavior.

The dataset undergoes comprehensive preprocessing, including data cleaning, feature engineering, exploratory data analysis, and visualization to understand customer characteristics and identify important patterns. Multiple supervised machine learning algorithms, including **Logistic Regression**, **Decision Tree**, **Random Forest**, and **XGBoost**, are implemented and evaluated using performance metrics such as Accuracy, Precision, Recall, F1-Score, ROC-AUC Score, and Confusion Matrix.

Comparative analysis of these models identifies the most effective algorithm for predicting credit card default. The project demonstrates how data analytics and machine learning techniques can assist financial institutions in minimizing credit risk while improving decision-making processes.

Keywords: Credit Card Default Prediction, Machine Learning, Data Analytics, Classification, Credit Risk, Logistic Regression, Random Forest, XGBoost.

TABLE OF CONTENTS

CERTIFICATE

DECLARATION

ACKNOWLEDGEMENT

ABSTRACT

LIST OF ABBREVIATIONS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1 INTRODUCTION

1.1 Background

1.2 Problem Statement

1.3 Objectives

1.4 Scope of the Project

CHAPTER 2 LITERATURE REVIEW

2.1 Introduction

2.2 Related Work

2.3 Research Gap

CHAPTER 3 DATASET AND METHODOLOGY

3.1 Dataset Source

3.2 Dataset Description

3.3 Feature Description

3.4 Workflow

3.5 Data Preprocessing

CHAPTER 4 DATA ANALYSIS AND PROCESSING

4.1 Exploratory Data Analysis

4.2 Data Visualization

4.3 Correlation Analysis

4.4 Feature Engineering

CHAPTER 5 MODEL DEVELOPMENT AND PERFORMANCE ANALYSIS

5.1 Logistic Regression

5.2 Decision Tree

5.3 Random Forest

5.4 XGBoost

5.5 Model Evaluation

5.6 Performance Comparison

CHAPTER 6 CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

6.2 Future Scope

REFERENCES

CHAPTER 1: INTRODUCTION

1.1 Background

In today's rapidly evolving financial sector, credit cards have become one of the most widely used financial instruments, offering customers flexibility in managing purchases and payments. However, with the increasing number of credit card users, financial institutions face significant challenges in managing credit risk due to customer payment defaults. A credit card default occurs when a customer fails to make the required payment within the specified time period, resulting in financial losses for banks and lending institutions.

Traditional credit risk assessment methods primarily rely on statistical models and manual evaluation techniques. Although these methods are simple and interpretable, they often fail to capture the complex relationships among customer demographics, repayment history, credit utilization, and financial behavior. As financial transactions continue to grow in volume and complexity, more intelligent approaches are required to accurately identify customers who are likely to default.

Machine Learning (ML) has emerged as a powerful solution for predictive analytics by enabling computers to learn patterns from historical data and make accurate predictions on unseen data. In the financial domain, machine learning algorithms can analyze large volumes of customer information, detect hidden patterns, and classify customers according to their likelihood of default. This allows financial institutions to make informed lending decisions, reduce financial losses, and improve overall risk management.

This internship project focuses on developing a machine learning-based framework for predicting credit card defaults using the **UCI Credit Card Default Dataset**. The project involves data preprocessing, exploratory data analysis (EDA), feature engineering, model development, performance evaluation, and interpretation of the results. Multiple classification algorithms are implemented and compared to identify the model that best predicts customer default behavior. The project demonstrates how data analytics and machine learning techniques can be effectively applied to solve real-world financial problems while supporting data-driven decision-making. This approach is consistent with the motivation and methodology discussed in your research paper, which emphasizes behavioral information and structured financial data for credit risk prediction.

1.2 Problem Statement

Credit card issuing institutions process thousands of customer applications every day. Identifying customers who are likely to default on future payments is a critical challenge, as inaccurate risk assessment can lead to substantial financial losses, reduced profitability, and increased operational risks.

Conventional credit scoring methods often depend on limited financial indicators and linear statistical models, making it difficult to capture the complex behavioral patterns associated with repayment history and credit utilization. Consequently, there is a need for a predictive system capable of analyzing multiple customer attributes simultaneously to improve the accuracy of default prediction.

The primary problem addressed in this project is to develop an intelligent machine learning model capable of predicting whether a credit card customer is likely to default on the next month's payment. The solution should effectively utilize customer demographic information, historical repayment records, billing amounts, payment history, and engineered behavioral features to classify customers into default and non-default categories. By providing accurate predictions, the system can assist financial institutions in minimizing credit risk, improving lending decisions, and supporting responsible financial management.

1.3 Objectives

The major objectives of this internship project are:

- To study the concept of credit risk assessment using machine learning techniques.
- To analyze the UCI Credit Card Default dataset and understand its attributes and characteristics.
- To perform data preprocessing by handling inconsistencies, feature scaling, and data transformation.
- To conduct Exploratory Data Analysis (EDA) to identify trends, correlations, and important customer behavior patterns.
- To engineer meaningful behavioral features that improve the representation of customer repayment behavior.
- To develop and compare multiple supervised machine learning models for credit card default prediction.
- To evaluate the performance of each model using appropriate classification metrics such as Accuracy, Precision, Recall, F1-score, and ROC-AUC.
- To identify the most effective predictive model for credit card default classification.

- To demonstrate the practical application of data analytics and machine learning techniques in financial risk management.

1.4 Scope

The scope of this internship project is focused on developing a machine learning-based predictive system for identifying potential credit card defaulters using historical customer data. The project covers the complete data analytics lifecycle, including data collection, preprocessing, exploratory data analysis, feature engineering, model training, evaluation, and interpretation of results.

The study utilizes the publicly available **UCI Credit Card Default Dataset**, which contains demographic information, repayment status, billing amounts, payment records, and credit limit information for 30,000 customers. Several supervised classification algorithms are implemented and compared to evaluate their effectiveness in predicting customer defaults.

The project is limited to offline predictive analysis using historical data and does not include real-time deployment, integration with banking systems, or live transaction monitoring. Additionally, the performance of the developed models depends on the quality and characteristics of the available dataset. Future enhancements may include real-time prediction systems, deep learning approaches, explainable artificial intelligence techniques, cloud deployment, and larger financial datasets to improve scalability and predictive performance.

CHAPTER 2: LITERATURE REVIEW

2.1 Existing Work

Several researchers have explored machine learning techniques for predicting credit card defaults, focusing on improving prediction accuracy, feature engineering, and model interpretability.

Yeh and Lien (2009) introduced the UCI Credit Card Default Dataset, which has become one of the most widely used benchmark datasets for credit risk prediction. Their study compared multiple data mining techniques and demonstrated that machine learning approaches can outperform conventional statistical methods in predicting customer defaults. This dataset has since been extensively adopted by researchers for evaluating classification algorithms and developing credit risk prediction systems.

Chang et al. (2018) applied Extreme Gradient Boosting (XGBoost) for financial risk assessment and reported that boosting algorithms effectively capture nonlinear relationships present in customer financial behavior. Compared with conventional models, XGBoost provided improved predictive performance by sequentially correcting errors made by previous decision trees.

Ke et al. (2017) proposed LightGBM, an efficient gradient boosting framework designed to improve training speed and reduce memory consumption while maintaining high prediction accuracy. Due to its leaf-wise tree growth strategy, LightGBM has become a popular choice for large-scale structured datasets.

Prokhorenkova et al. (2018) developed CatBoost, a gradient boosting algorithm specifically designed to handle categorical variables efficiently while reducing overfitting through ordered boosting techniques. CatBoost has demonstrated excellent performance on structured financial datasets containing categorical customer attributes.

Lundberg and Lee (2017) introduced SHAP (SHapley Additive exPlanations), an explainable AI technique that quantifies the contribution of each feature toward model predictions. SHAP has become a standard approach for interpreting complex ensemble models and increasing transparency in machine learning systems used for financial decision-making.

Recent studies have shifted their focus from merely comparing algorithms to developing **behavior-oriented features** that summarize customer repayment

patterns over time. Instead of considering each monthly transaction independently, researchers aggregate repayment behavior, credit utilization, payment consistency, and billing information into meaningful behavioral indicators. Such features often improve model performance while providing better insight into customer financial habits.

The conference paper developed during this project extends this direction by introducing eight engineered behavioral features, including average payment status, maximum repayment delay, payment ratio, total payments, and credit utilization. These features summarize long-term customer behavior rather than relying solely on raw monthly account information. The paper further combines these engineered features with modern gradient boosting algorithms and explainability techniques such as SHAP to build an interpretable credit default prediction framework.

Most existing studies evaluate multiple supervised learning algorithms including:

- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine (SVM)
- XGBoost
- LightGBM
- CatBoost

Performance is generally assessed using evaluation metrics such as Accuracy, Precision, Recall, F1-Score, ROC-AUC, and Confusion Matrix, providing a comprehensive comparison of classification effectiveness.

2.2 Research Gap

Although considerable research has been conducted on credit card default prediction, several limitations still exist in the current literature.

Many earlier studies focus primarily on improving prediction accuracy without investigating whether the additional engineered features provide statistically meaningful improvements. Several researchers report only single train-test split results, making it difficult to determine whether the observed improvements are robust across different data partitions.

Another limitation is that many machine learning models function as "black boxes," providing limited explanation of how predictions are generated. In financial applications, interpretability is essential because lending decisions must often satisfy regulatory requirements and be understandable to stakeholders.

Furthermore, many previous studies compare machine learning algorithms using only the original dataset attributes while overlooking behavioral aggregation techniques that summarize repayment consistency, credit utilization, and payment behavior over time. Although recent research has begun exploring engineered behavioral features, relatively few studies combine feature engineering, advanced ensemble learning, explainable AI, and systematic model evaluation within a unified framework.

Based on these observations, the present internship project addresses the following research gaps:

- Development of a comprehensive data analytics pipeline from preprocessing to model evaluation.
- Integration of behavioral feature engineering with traditional customer attributes.
- Comparative analysis of multiple supervised machine learning algorithms.
- Application of explainable machine learning concepts to improve model transparency.
- Evaluation using multiple performance metrics instead of relying on accuracy alone.
- Practical demonstration of how machine learning techniques can support credit risk assessment in financial institutions.

Unlike many research-oriented studies, this internship project emphasizes both **technical implementation** and **practical applicability**, demonstrating how machine learning can be integrated into a complete data analytics workflow for credit card default prediction. This aligns with the motivation in your conference paper, which highlights behavioral feature engineering, explainability, and rigorous evaluation while acknowledging the need for careful interpretation of improvements.

CHAPTER 3: DATASET & METHODOLOGY

3.1 Dataset Source

The dataset used in this internship project is the **Default of Credit Card Clients Dataset**, publicly available from the **UCI Machine Learning Repository**. It is one of the most widely used benchmark datasets for evaluating machine learning algorithms in credit risk prediction and financial analytics.

The dataset was originally developed by **Yeh and Lien (2009)** to compare the predictive performance of various data mining techniques for estimating the probability of credit card default. It contains historical financial information and repayment records of credit card holders in Taiwan and is extensively used in academic research and industrial applications related to credit risk assessment. The conference paper associated with this project also adopts the same dataset as the foundation for model development and behavioral feature engineering.

The dataset consists of customer demographic information, credit limits, repayment status over six months, monthly bill amounts, monthly payment amounts, and a binary target variable indicating whether a customer defaulted on the next month's payment.

Dataset Details

Attribute	Description
Dataset Name	Default of Credit Card Clients
Source	UCI Machine Learning Repository
Domain	Banking and Finance
Total Records	30,000
Original Features	23
Target Variable	default.payment.next.month

Target Classes	0 = Non-default, 1 = Default
----------------	------------------------------

3.2 Dataset Description

The dataset contains **30,000 customer records** with **23 independent attributes** and **one target variable**. The attributes represent customer demographics, financial history, repayment behavior, billing amounts, and payment records over a six-month period.

The target variable **default.payment.next.month** indicates whether the customer defaulted on the next month's payment.

Dataset Property	Value
Number of Records	30,000
Number of Features	23
Target Variable	default.payment.next.month
Type of Problem	Binary Classification
Missing Values	None
Duplicate Records	Checked and Removed (if any)

Description of Important Features

Feature	Description
ID	Unique customer identification number
LIMIT_BAL	Amount of credit limit assigned to the customer
SEX	Gender (1 = Male, 2 = Female)
EDUCATION	Education level
MARRIAGE	Marital status
AGE	Customer age
PAY_0 – PAY_6	Repayment status for previous six months
BILL_AMT1 – BILL_AMT6	Monthly bill statements

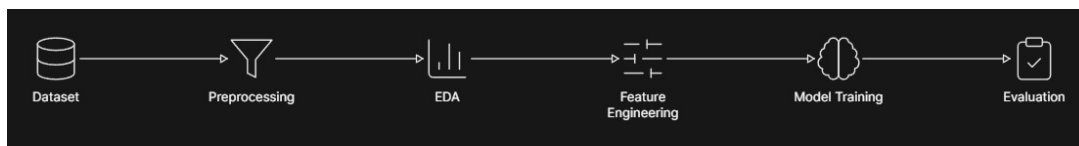
PAY_AMT1 – PAY_AMT6	Monthly payment amounts
default.payment.next.month	Target variable indicating default (1) or non-default (0)

The dataset exhibits a moderate class imbalance, with a larger proportion of non-default customers than default customers. Such imbalance is common in financial datasets and must be considered during model evaluation to avoid misleading accuracy values.

3.3 Workflow

The methodology adopted follows a structured data analytics pipeline, beginning with data collection and ending with model evaluation. Each stage contributes to building an accurate and reliable predictive model for identifying potential credit card defaulters.

The overall workflow of the project is illustrated below.



Data Collection

The dataset was obtained from the UCI Machine Learning Repository and imported into the Python environment for further analysis.

Data Preprocessing

The raw dataset was cleaned, verified for inconsistencies, and prepared for machine learning.

Exploratory Data Analysis

Various statistical summaries and visualizations were generated to understand customer behavior and identify important relationships among variables.

Feature Engineering

Behavioral features were created by aggregating repayment history, payment amounts, and billing information to better represent customer financial behavior. The behavioral feature engineering approach is inspired by the methodology presented in your conference paper.

Model Development

Multiple supervised machine learning algorithms were trained and compared for predicting customer defaults.

Model Evaluation

The trained models were evaluated using classification metrics such as Accuracy, Precision, Recall, F1-score, ROC-AUC Score, and Confusion Matrix.

3.4 Data Preprocessing

Data preprocessing is one of the most important stages in any machine learning project, as the quality of the input data directly affects the performance of predictive models. The preprocessing steps performed in this project are described below.

Missing Value Analysis

The dataset was examined for missing or null values using the `isnull()` function. No missing values were found in the dataset; therefore, no imputation techniques were required.

Duplicate Record Analysis

The dataset was checked for duplicate records. Duplicate observations, if present, were removed to ensure data integrity and avoid bias during model training.

Train-Test Split

Finally, the processed dataset was divided into training and testing subsets using an **80:20 ratio**. Stratified sampling was employed to preserve the proportion of default and non-default customers in both subsets, ensuring a fair evaluation of model performance. This approach matches the experimental setup described in your research paper.

CHAPTER 4: EXPLORATORY DATA ANALYSIS

4.1 Dataset Overview

Before any modelling was carried out, the cleaned dataset was inspected using `df.info()` and `df.describe()` to verify data types, non-null counts, and the statistical spread of every numeric attribute. Table 4.1 summarizes the overall shape and composition of the dataset used for exploratory analysis and model development.

Property	Value
Total records	30,000
Original attributes (excluding ID and target)	23
Attributes after behavioral feature engineering	33
Target variable	default.payment.next.month (DEFAULT)
Non-default cases (class 0)	23,364 (77.9%)
Default cases (class 1)	6,636 (22.1%)
Missing values	None
Duplicate records	Checked and removed
Training set (80%, stratified)	24,000 records
Testing set (20%, stratified)	6,000 records

Figure 4.0: Dataset overview and train/test partitioning

```
# Define the complete path to the raw CSV file.
RAW_DATA_PATH = os.path.join(
    RAW_DATA_DIR,
    "UCI_Credit_Card.csv"
)

# Load the CSV file into a Pandas DataFrame.
df_raw = pd.read_csv(RAW_DATA_PATH)

# Display the first five rows.
display(df_raw.head())
```

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	PAY_5	PAY_6	BILL_AMT1	BILL_AMT2	BILL_AMT3	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	PAY_AMT2
0	1	20000.0	2	2	1	24	2	2	-1	-1	-2	-2	3913.0	3102.0	689.0	0.0	0.0	0.0	0.0	689.0
1	2	120000.0	2	2	2	26	-1	2	0	0	0	2	2682.0	1725.0	2682.0	3272.0	3455.0	3261.0	0.0	1000.0
2	3	90000.0	2	2	2	34	0	0	0	0	0	0	29239.0	14027.0	13559.0	14331.0	14948.0	15549.0	1518.0	1500.0
3	4	50000.0	2	2	1	37	0	0	0	0	0	0	46990.0	48233.0	49291.0	28314.0	28959.0	29547.0	2000.0	2019.0
4	5	50000.0	1	2	1	57	-1	0	-1	0	0	0	8617.0	5670.0	35835.0	20940.0	19146.0	19131.0	2000.0	36681.0

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    stratify=y,
    random_state=RANDOM_STATE
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

print("\nTraining target distribution:")
print(y_train.value_counts(normalize=True))

print("\nTesting target distribution:")
print(y_test.value_counts(normalize=True))
```

```

... Training samples: 23972
    Testing samples: 5993

Training target distribution:
DEFAULT
0    0.778742
1    0.221258
Name: proportion, dtype: float64

Testing target distribution:
DEFAULT
0    0.778742
1    0.221258
Name: proportion, dtype: float64

```

Figure 4.1: Output of `df.info()` showing column data types and non-null counts

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 25 columns):
 #   Column                                     Non-Null Count  Dtype
---  -
 0   ID                                         30000 non-null  int64
 1   LIMIT_BAL                                 30000 non-null  float64
 2   SEX                                         30000 non-null  int64
 3   EDUCATION                                 30000 non-null  int64
 4   MARRIAGE                                 30000 non-null  int64
 5   AGE                                         30000 non-null  int64
 6   PAY_0                                       30000 non-null  int64
 7   PAY_2                                       30000 non-null  int64
 8   PAY_3                                       30000 non-null  int64
 9   PAY_4                                       30000 non-null  int64
10  PAY_5                                       30000 non-null  int64
11  PAY_6                                       30000 non-null  int64
12  BILL_AMT1                                 30000 non-null  float64
13  BILL_AMT2                                 30000 non-null  float64
14  BILL_AMT3                                 30000 non-null  float64
15  BILL_AMT4                                 30000 non-null  float64
16  BILL_AMT5                                 30000 non-null  float64
17  BILL_AMT6                                 30000 non-null  float64
18  PAY_AMT1                                 30000 non-null  float64
19  PAY_AMT2                                 30000 non-null  float64
20  PAY_AMT3                                 30000 non-null  float64
21  PAY_AMT4                                 30000 non-null  float64
22  PAY_AMT5                                 30000 non-null  float64
23  PAY_AMT6                                 30000 non-null  float64
24  default.payment.next.month               30000 non-null  int64
dtypes: float64(13), int64(12)
memory usage: 5.7 MB

```

4.2 Data Visualization

A series of univariate and bivariate visualizations were generated to understand customer demographics, credit behavior, and their relationship with the default outcome. Each figure below corresponds to a plotting cell in the Raw Data Analysis Notebook.

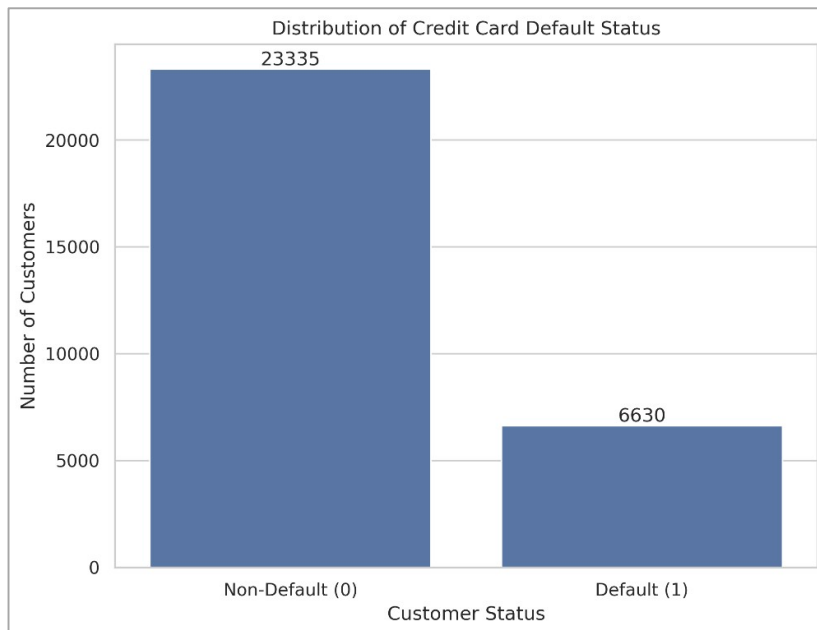


Figure 4.3: Target class distribution — count of default vs. non-default customers

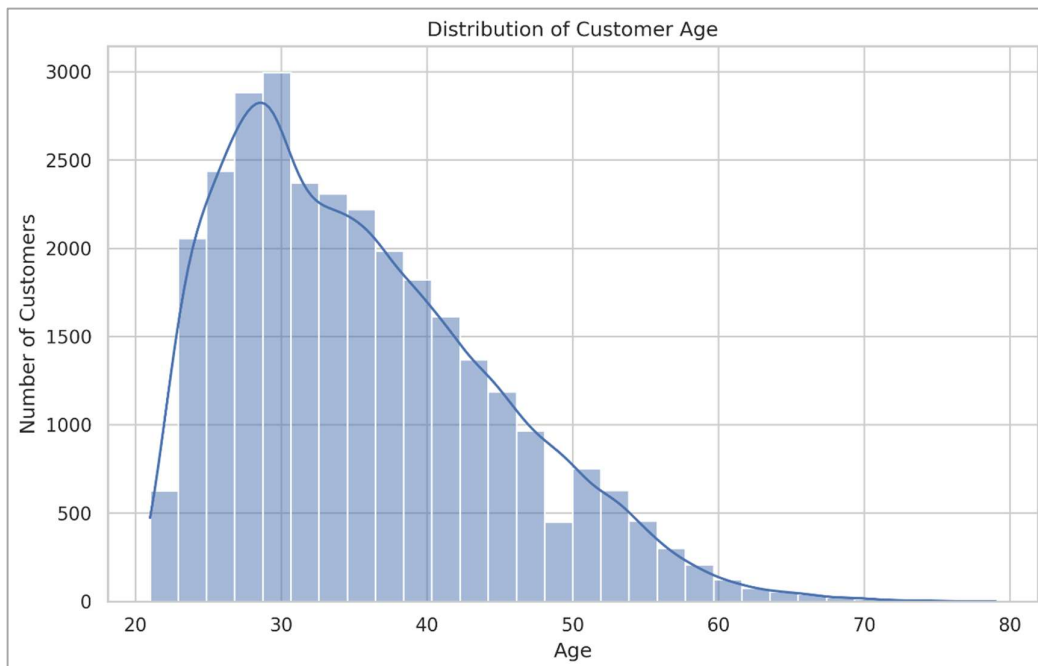


Figure 4.4: Age distribution of customers

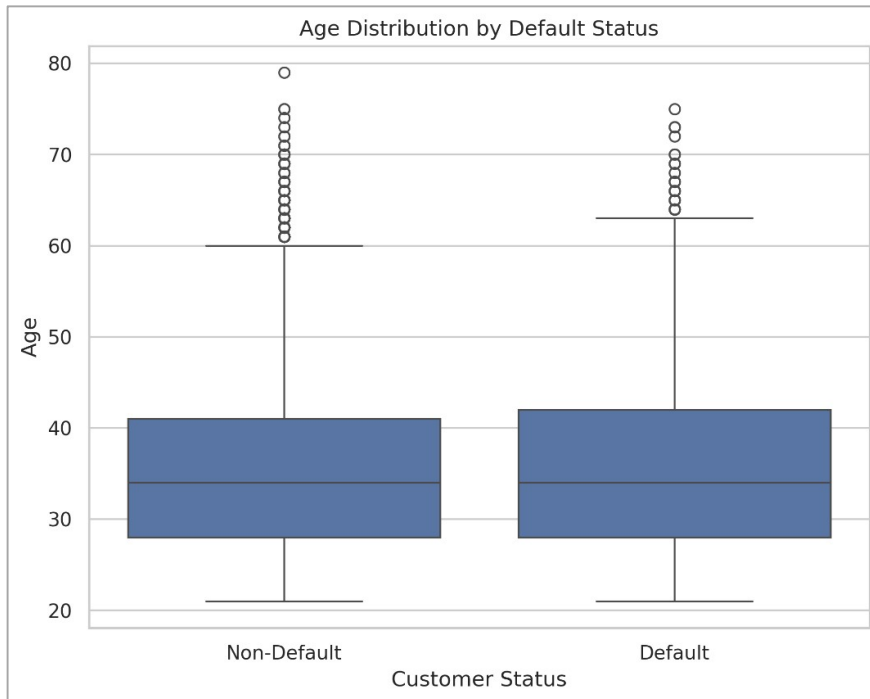
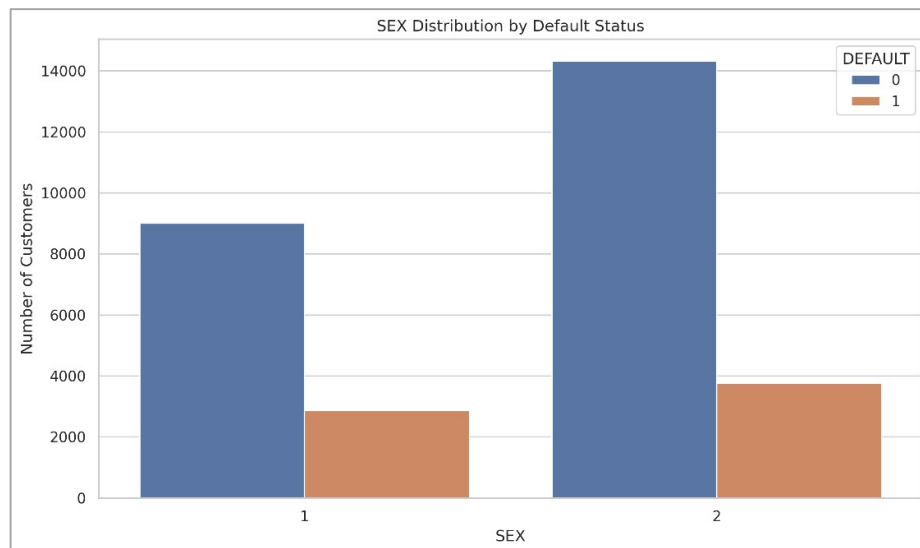


Figure 4.5: Age distribution grouped by default status



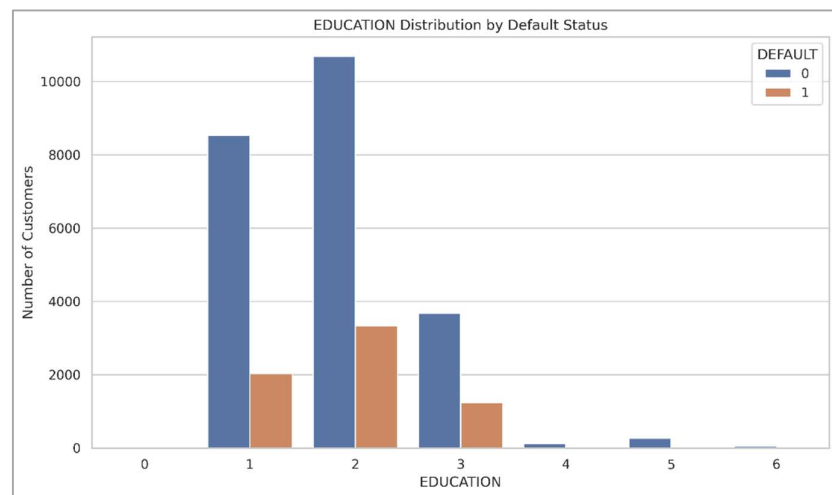
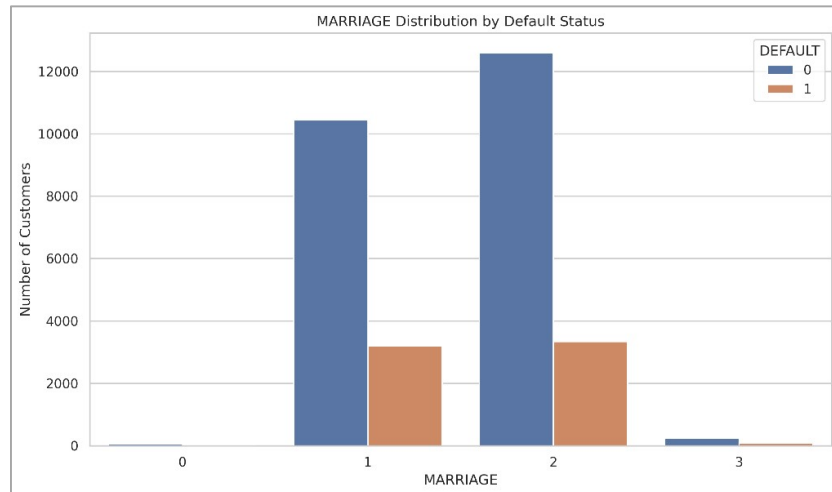


Figure 4.6: Distribution of demographic features (Sex, Education, Marriage)

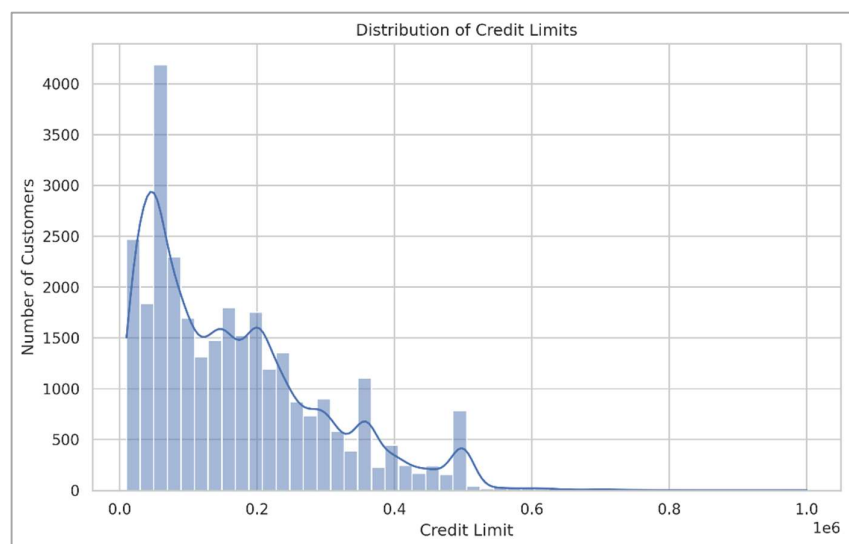
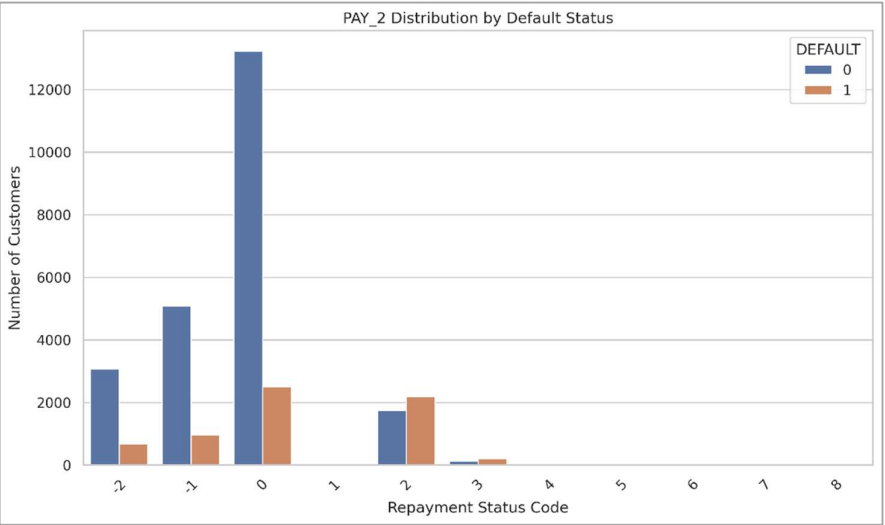
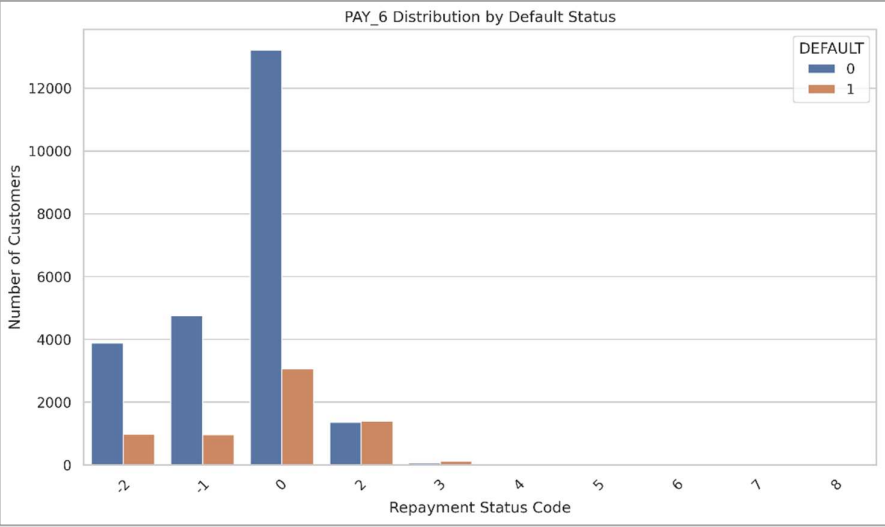
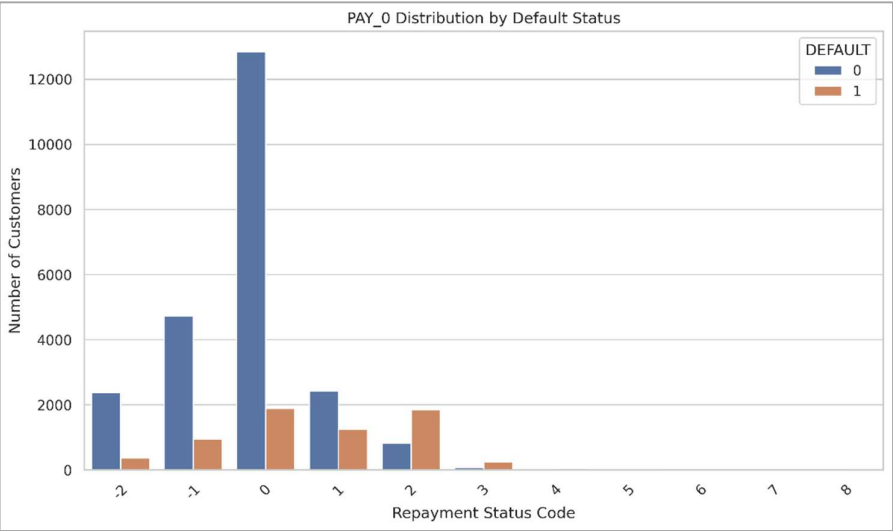


Figure 4.7: Credit limit (LIMIT_BAL) distribution



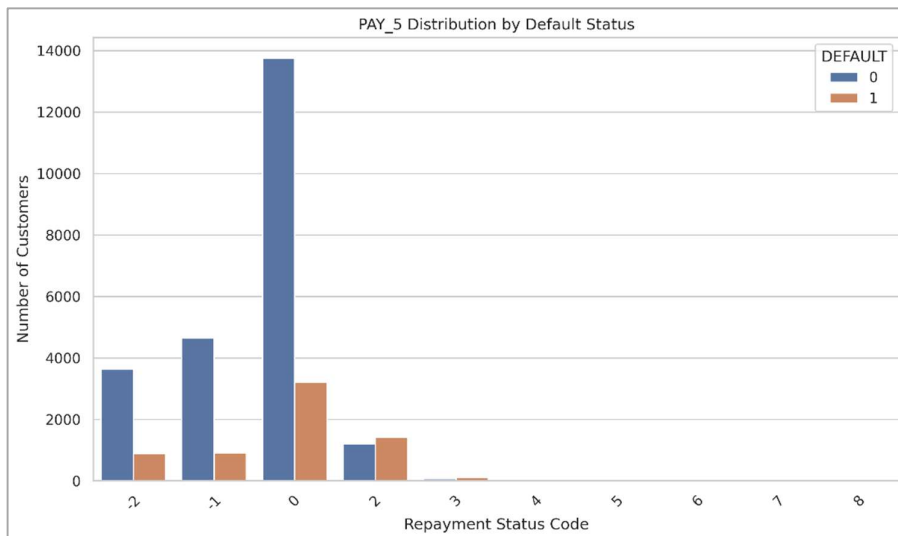
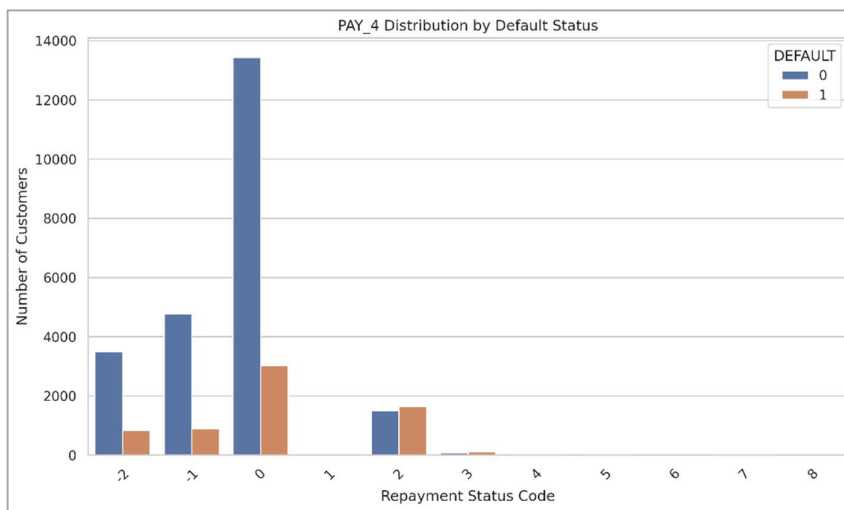
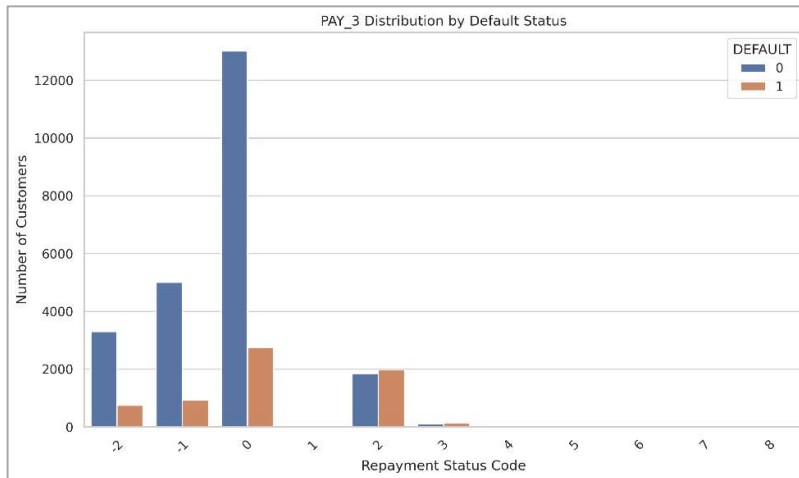
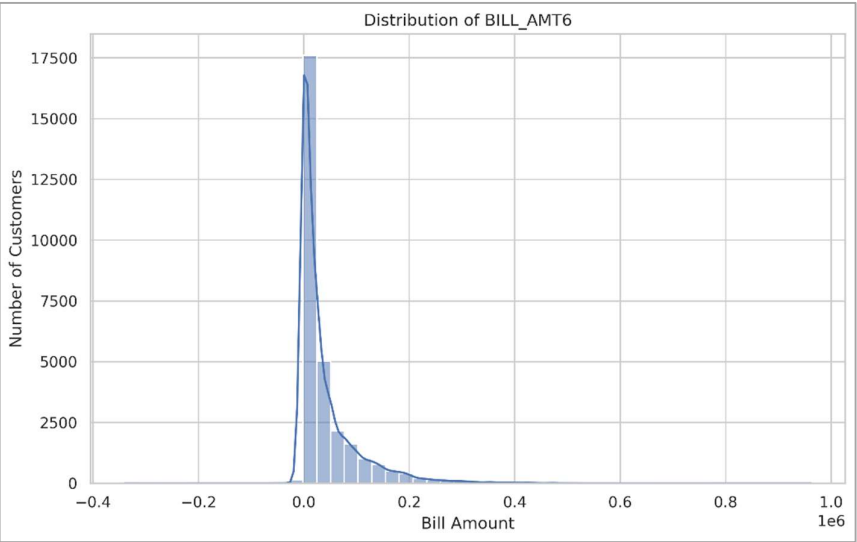
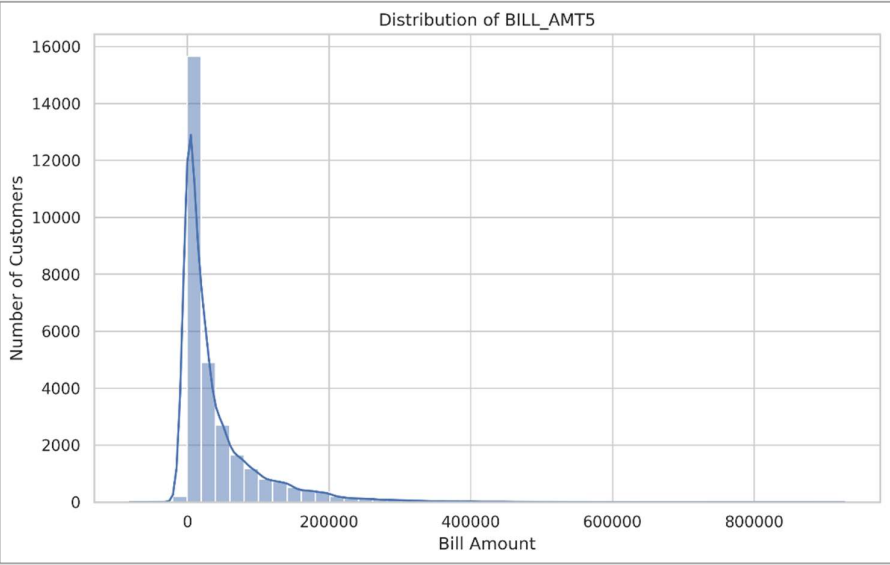
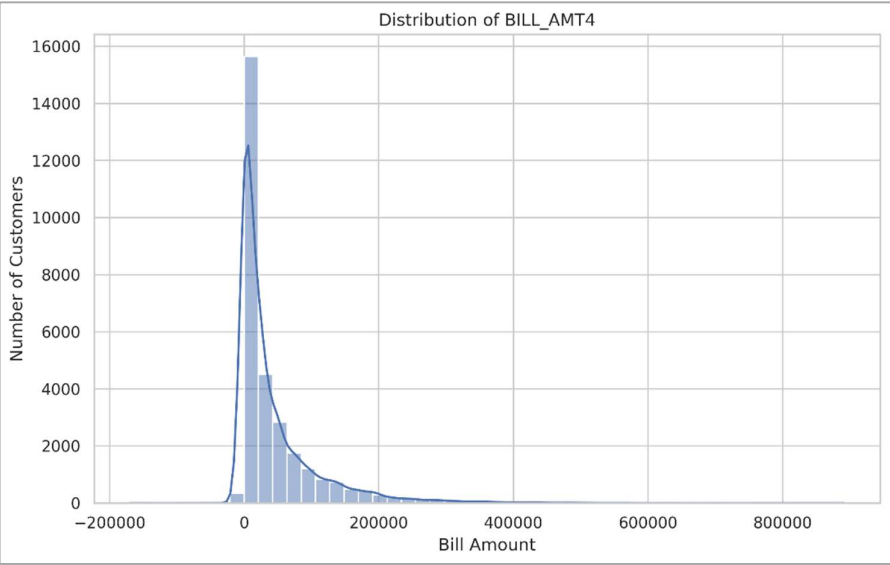


Figure 4.8: Repayment status (PAY_0 – PAY_6) distributions



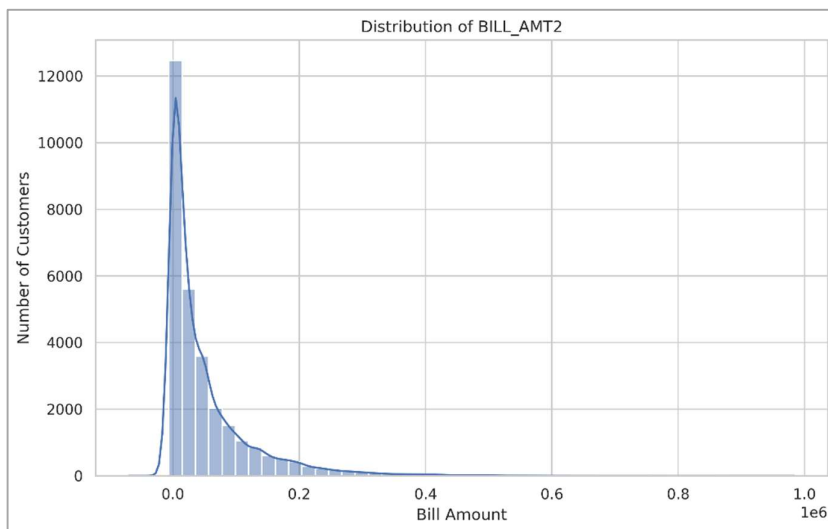
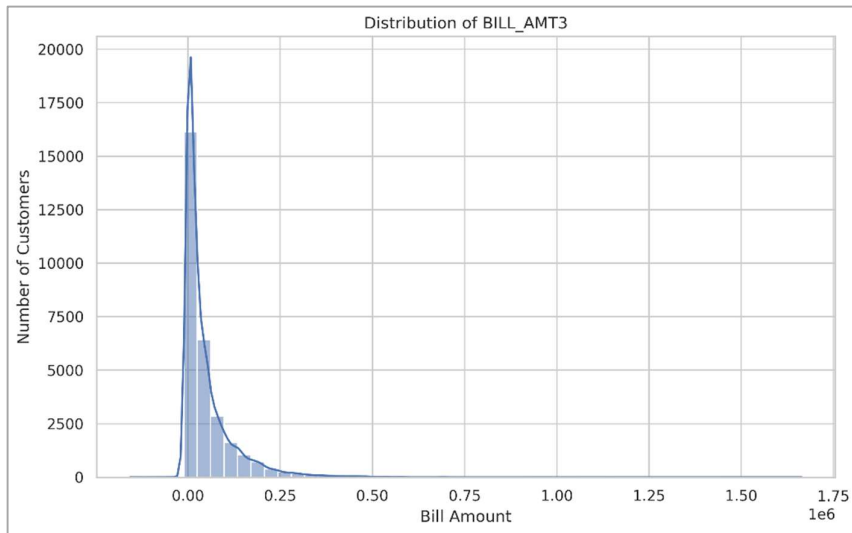
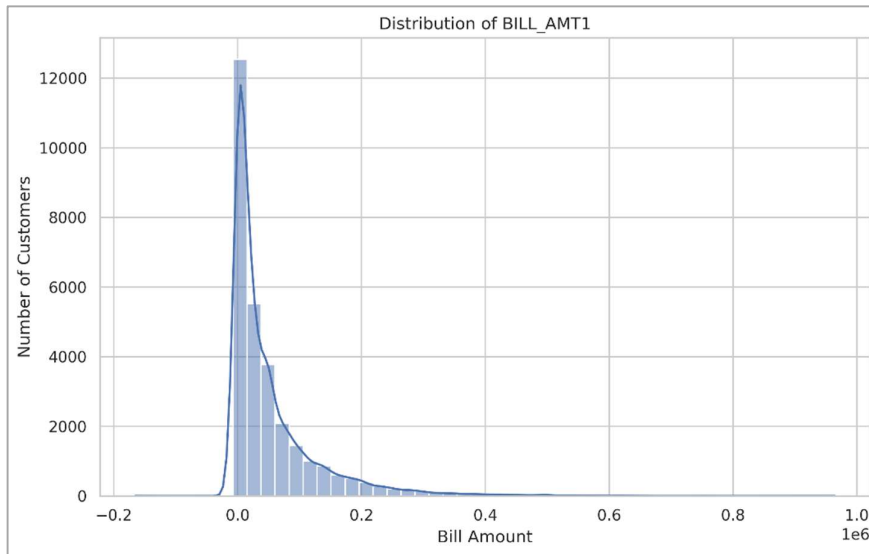
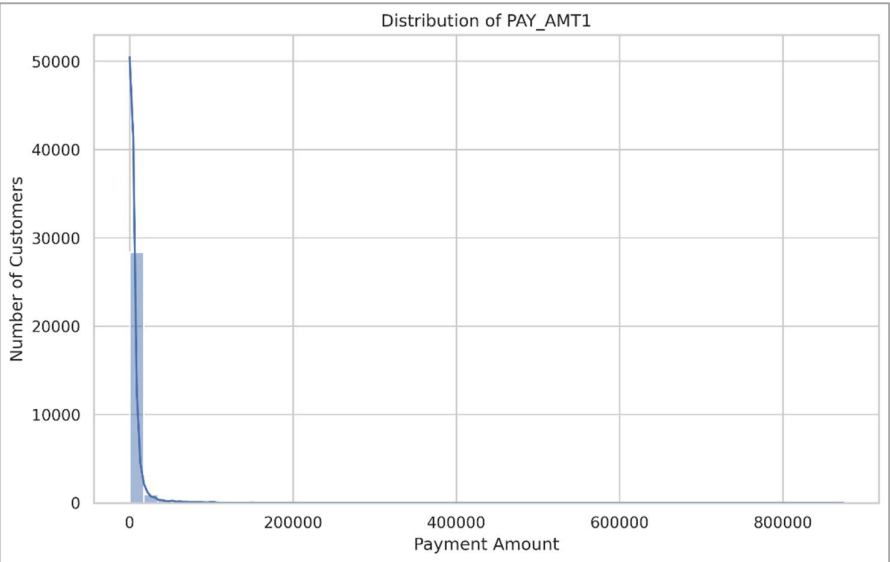
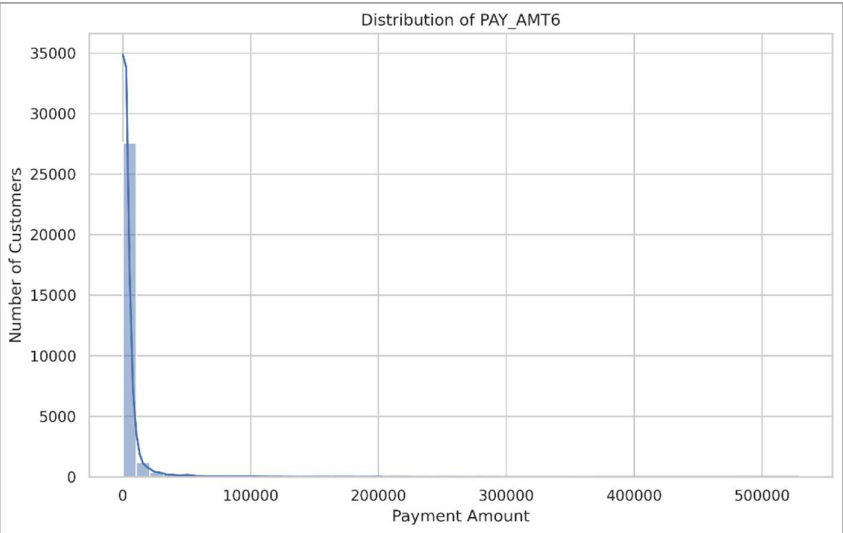
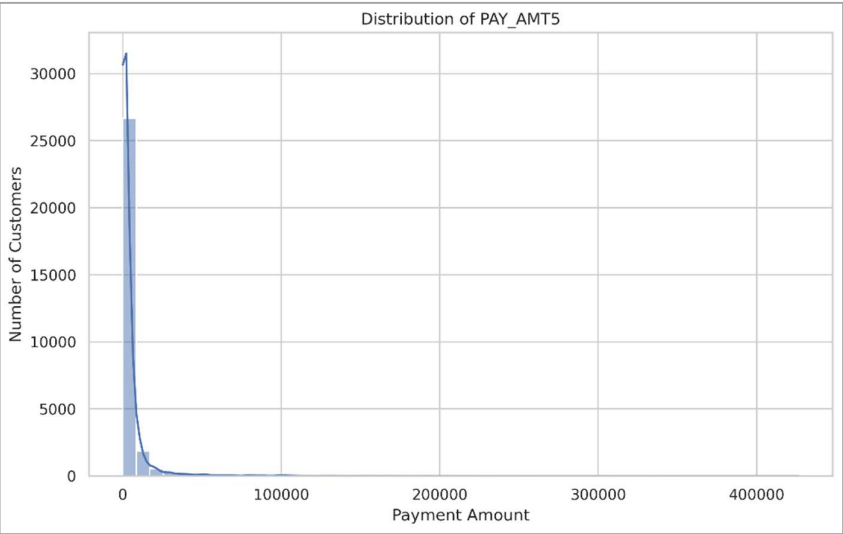


Figure 4.10: Bill amount (BILL_AMT1 – BILL_AMT6) distributions



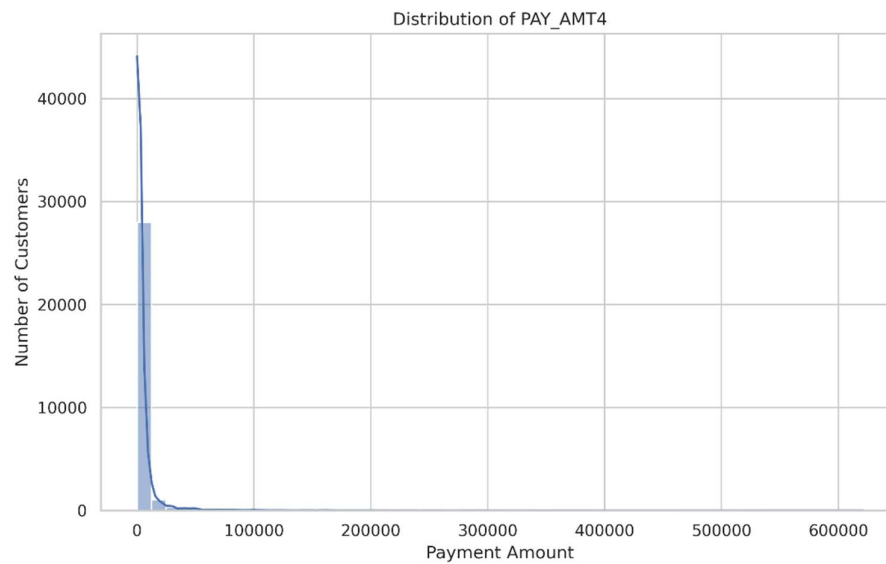
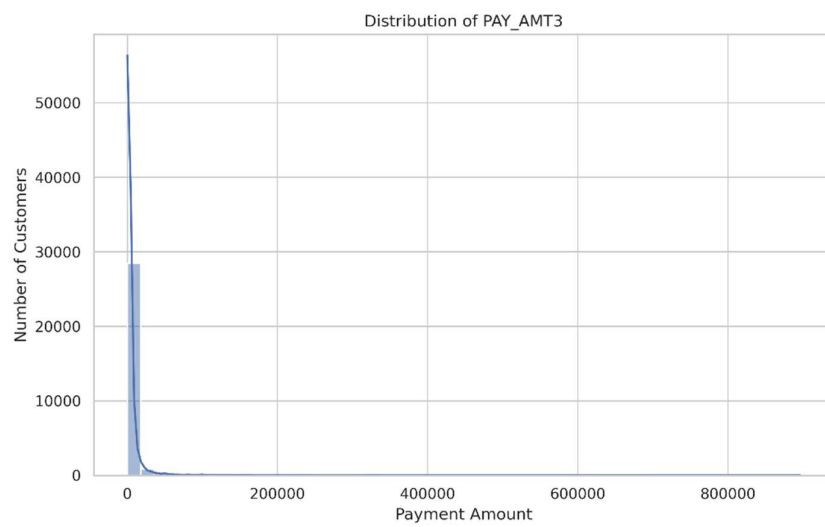
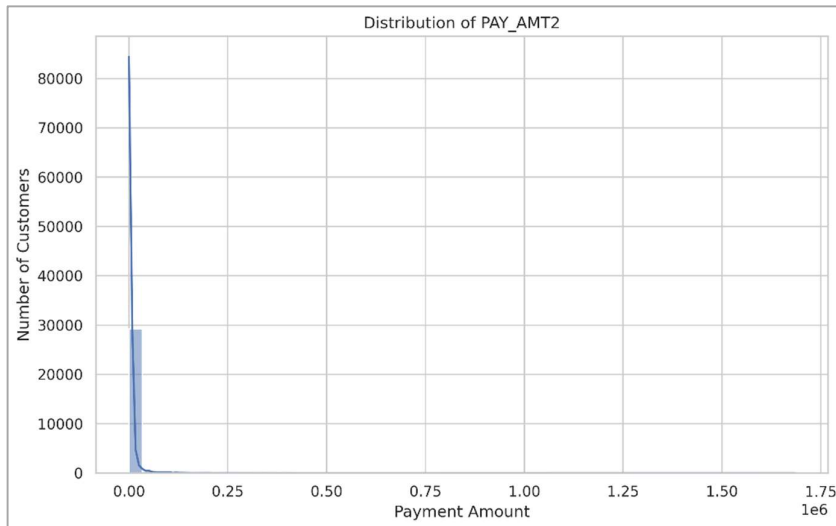


Figure 4.11: Payment amount (PAY_AMT1 – PAY_AMT6) distributions

4.3 Correlation Analysis

Pairwise Pearson correlation of each original attribute with the binary target variable was examined prior to feature engineering. The six repayment-status columns (PAY_0 – PAY_6) were found to be the strongest positively correlated original attributes ($r = 0.20 - 0.33$, led by PAY_0), while LIMIT_BAL was the strongest negatively correlated attribute ($r \approx -0.15$). The payment-amount columns showed a weaker negative correlation with default. This pattern directly motivated the behavioral feature engineering strategy described in Section 4.5 — aggregating the six monthly repayment-status codes into summary statistics consolidates their shared signal into a smaller number of more robust predictors.

Figure 4.12: Correlation heatmap of all numeric features

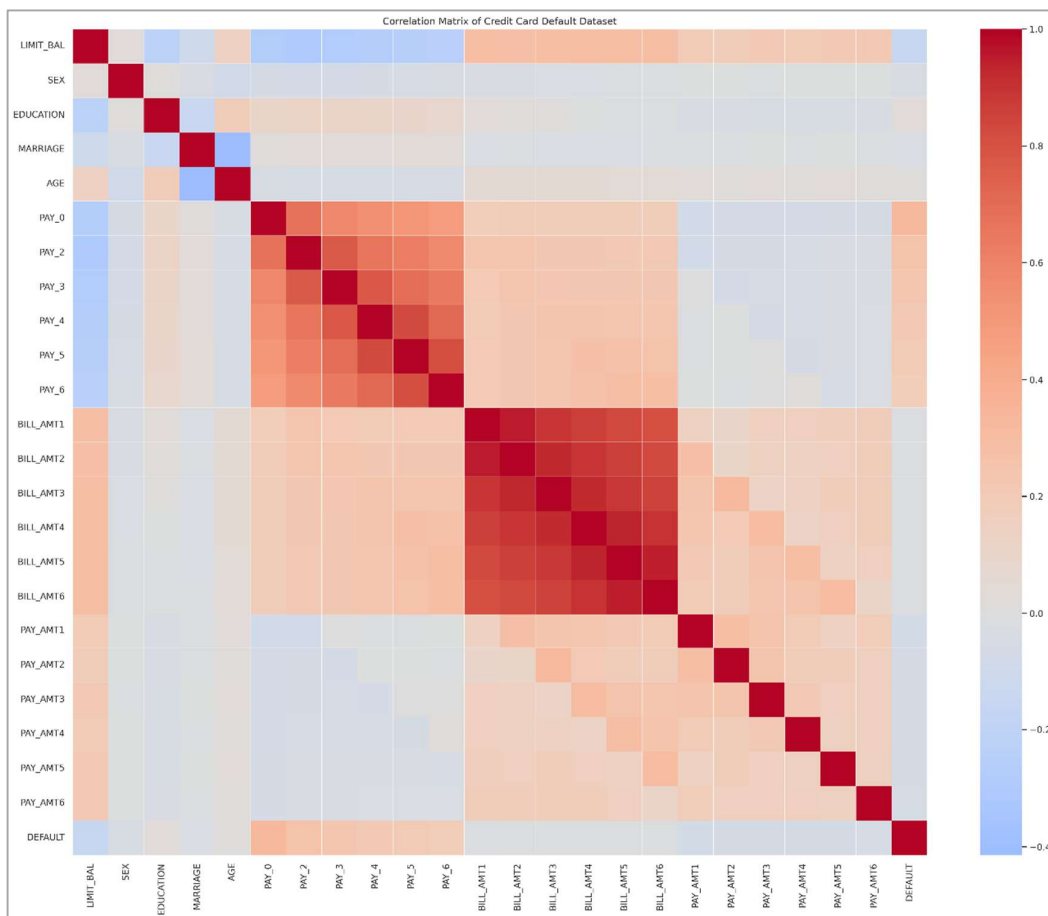
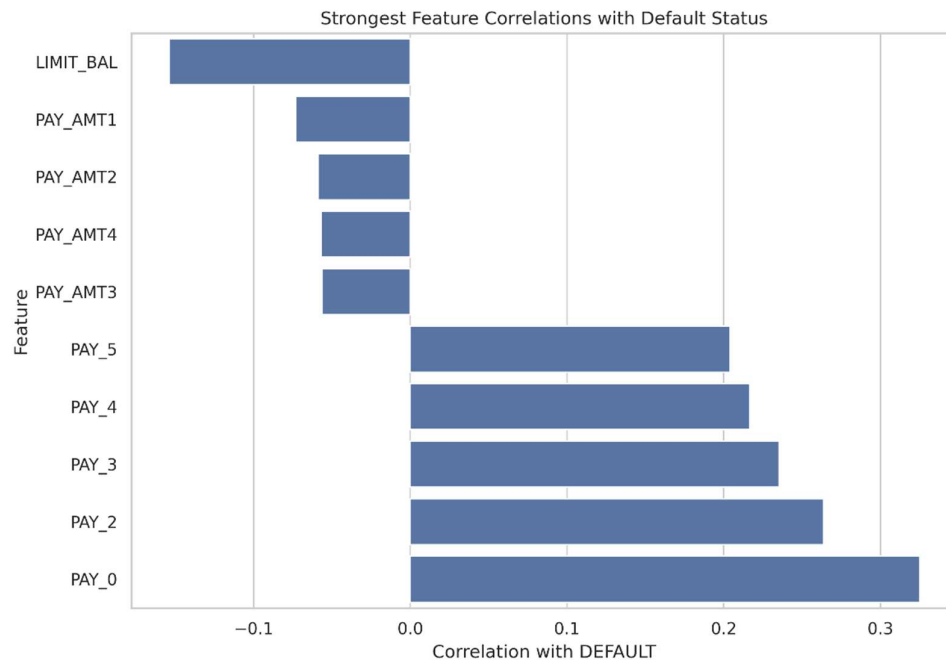


Figure 4.13: Top 10 positive and negative feature correlations with the target variable



4.4 Outlier Analysis

Outlier analysis was performed on the bill amount and payment amount columns using the standard $1.5 \times \text{IQR}$ (Interquartile Range) method. Approximately 8–10% of values in these columns were found to lie beyond the $1.5 \times \text{IQR}$ threshold, which is consistent with the heavy-tailed nature of financial transaction data. These extreme values were retained rather than removed, since unusually large balances and payments are themselves informative of customer risk rather than being data-entry errors.

```
# =====  
# RUN OUTLIER ANALYSIS  
# =====  
  
# Combine bill and payment features.  
financial_amount_features = (  
    bill_features +  
    payment_features  
)  
  
# Generate the outlier summary.  
outlier_summary = calculate_iqr_outlier_summary(  
    df,  
    financial_amount_features  
)  
  
# Round values for readability.  
outlier_summary = outlier_summary.round(2)  
  
# Display the summary.  
display(outlier_summary)  
  
# Save the table to Google Drive.  
outlier_summary.to_csv(  
    os.path.join(  
        TABLES_DIR,  
        "outlier_analysis_summary.csv"  
    ),  
    index=False  
)  
  
print("Outlier analysis saved successfully.")
```

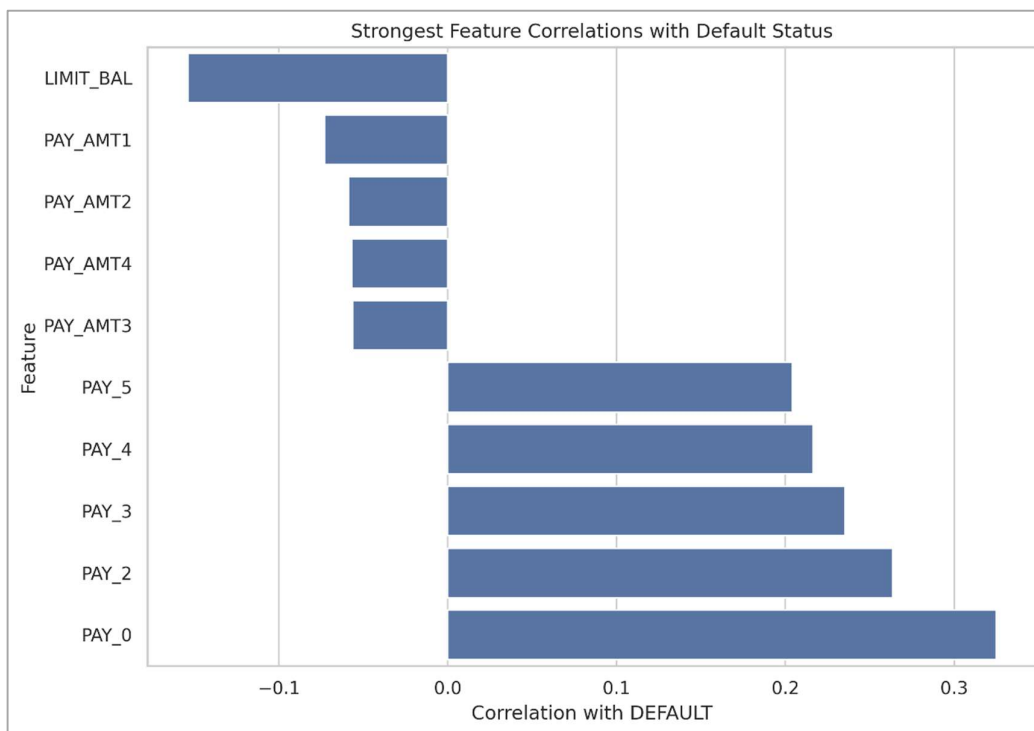
Code Snippet 4.1: Outlier detection using the IQR method on bill and payment amount columns

4.5 Feature Engineering

Ten behavior-oriented features were engineered from the six-month repayment history and appended to the 23 original attributes, forming a 33-feature representation used for model training. These variables summarize repayment consistency, credit utilization patterns, and payment-related indicators, capturing behavioral trends over the six-month observation window that are not visible in any single month's snapshot.

Feature	Description
AVG_BILL_AMT	Mean outstanding bill amount across the six-month window
AVG_PAY_AMT	Mean monthly payment amount across the six-month window
AVG_PAY_STATUS	Mean repayment-delay status across the six months
MAX_PAY_STATUS	Worst (maximum) repayment-delay status observed in any month
TOTAL_BILL	Sum of bill amounts across all six months
TOTAL_PAYMENT	Sum of payment amounts across all six months
CREDIT_UTILIZATION	Average bill amount relative to the assigned credit limit
PAYMENT_RATIO	Ratio of payment amount to bill amount, averaged across months
DELAYED_MONTHS	Count of months (of six) with a delayed repayment status ($PAY_n > 0$)
SEVERE_DELAY_MONTHS	Count of months with a severe repayment delay ($PAY_n \geq 2$)

Table 4.2: Engineered behavioral feature set (10 features)



Snippet 4.2: Behavioral feature engineering code (AVG_PAY_STATUS, CREDIT_UTILIZATION, DELAYED_MONTHS, etc.)

4.6 Key Findings

- Class imbalance:** The dataset is moderately imbalanced, with non-default customers (77.9%) far outnumbering default customers (22.1%), which was considered during model evaluation.

- **PAY_0 dominance:** The most recent repayment status (PAY_0) shows the strongest correlation with default among all raw attributes, confirming that recent repayment behavior is the single most direct signal of risk.
- **Behavioral patterns:** Customers with higher credit utilization and a greater number of delayed months tend to show a visibly higher default rate in the bivariate plots.
- **Skewed financial fields:** Bill and payment amounts are heavily right-skewed with a meaningful share of outlier values, which were retained as they carry genuine risk information.
- **Data quality:** No missing values were present, and the engineered behavioral features show a coherent correlation structure with the original account attributes, validating their use as additional predictors.

CHAPTER 5: MODEL DEVELOPMENT & PERFORMANCE ANALYSIS

5.1 Machine Learning Algorithms

Six supervised classification algorithms were implemented and compared on the 33-feature engineered representation (23 original attributes + 10 engineered behavioral features):

- **Logistic Regression:** A linear baseline model using a StandardScaler pipeline, chosen for its transparency and widespread use in traditional credit scoring.
- **Decision Tree:** A single tree-based classifier used as a non-linear baseline.
- **Random Forest:** A bagging ensemble of decision trees that reduces variance relative to a single tree.
- **XGBoost:** A gradient boosting framework that sequentially corrects the errors of previous trees.
- **LightGBM:** A leaf-wise gradient boosting framework designed for training speed and efficiency on large structured datasets.
- **CatBoost:** A gradient boosting algorithm with ordered boosting, selected for its strong handling of structured/categorical financial data.

```
baseline_results = []

for model_name, model in models.items():

    print(f"Evaluating {model_name}...")

    # Perform five-fold cross-validation.
    scores = cross_validate(
        model,
        X_train,
        y_train,
        cv=cv,
        scoring=scoring,
        n_jobs=-1
    )

    # Store mean cross-validation results.
    baseline_results.append({

        "Model": model_name,

        "Accuracy": scores[
```

```

        "test_accuracy"
    ].mean(),

    "Precision": scores[
        "test_precision"
    ].mean(),

    "Recall": scores[
        "test_recall"
    ].mean(),

    "F1-Score": scores[
        "test_f1"
    ].mean(),

    "ROC-AUC": scores[
        "test_roc_auc"
    ].mean(),

    "Accuracy Std": scores[
        "test_accuracy"
    ].std()
})

# Convert results to a DataFrame.
baseline_results_df = pd.DataFrame(
    baseline_results
)

# Sort by accuracy.
baseline_results_df = baseline_results_df.sort_values(
    by="Accuracy",
    ascending=False
).reset_index(drop=True)

display(baseline_results_df)

```

Code Snippet 5.1: Model pipeline definitions for the six baseline classifiers

5.2 Training Process

The dataset was split into training (24,000 records) and testing (6,000 records) subsets using stratified sampling with a fixed random seed, preserving the 77.9% : 22.1% class ratio in both subsets. All six models were evaluated using five-fold stratified cross-validation to obtain a robust estimate of performance across folds rather than a single train/test split. The top-performing baseline model, CatBoost,

was further tuned using Optuna, a Bayesian hyperparameter optimization framework, and a threshold-optimization sweep (0.30 – 0.70) was run to study the effect of the decision threshold on precision and recall before the standard 0.5 threshold was adopted for final reporting. LightGBM was tuned in the same way for comparison.

```
# =====  
  
# FIVE-FOLD CV  
  
# ORIGINAL VS ENGINEERED FEATURES  
  
# =====  
  
original_cv_scores = []  
engineered_cv_scores = []  
  
fold_results = []  
  
# Use the existing five-fold stratified CV object.  
  
for fold_number, (  
    train_indices,  
    validation_indices  
) in enumerate(  
    cv.split(  
        X_train,  
        y_train  
    ),  
    start=1  
):  
  
    # -----  
  
    # ORIGINAL FEATURES  
  
    # -----
```

```
X_original_train_fold = (  
    X_original_train.iloc[  
        train_indices  
    ]  
)  
  
X_original_val_fold = (  
    X_original_train.iloc[  
        validation_indices  
    ]  
)  
  
# -----  
  
# ENGINEERED FEATURES  
  
# -----  
  
X_engineered_train_fold = (  
    X_train.iloc[  
        train_indices  
    ]  
)  
  
X_engineered_val_fold = (  
    X_train.iloc[  
        validation_indices  
    ]  
)  
  
# Target values.  
  
y_train_fold = (  
    y_train.iloc[  
        train_indices
```

```

    ]
)
y_val_fold = (
    y_train.iloc[
        validation_indices
    ]
)
# -----
# Train original-feature CatBoost
# -----

original_model = CatBoostClassifier(
    iterations=300,
    learning_rate=0.05,
    depth=6,
    verbose=False,
    random_seed=RANDOM_STATE
)

original_model.fit(
    X_original_train_fold,
    y_train_fold
)

original_probabilities = (
    original_model
    .predict_proba(
        X_original_val_fold
    )[:, 1]
)

```

```
original_auc = roc_auc_score(
    y_val_fold,
    original_probabilities
)

# -----

# Train engineered-feature CatBoost
# -----

engineered_model = CatBoostClassifier(
    iterations=300,
    learning_rate=0.05,
    depth=6,
    verbose=False,
    random_seed=RANDOM_STATE
)

engineered_model.fit(
    X_engineered_train_fold,
    y_train_fold
)

engineered_probabilities = (
    engineered_model
    .predict_proba(
        X_engineered_val_fold
    )[:, 1]
)

engineered_auc = roc_auc_score(
    y_val_fold,
```

```

        engineered_probabilities

    )

    # Store fold results.

    fold_results.append({

        "Fold": fold_number,

        "Original ROC-AUC": original_auc,

        "Engineered ROC-AUC": engineered_auc,

        "Difference": (

            engineered_auc -

            original_auc

        )

    })

# Convert fold results to DataFrame.

table_fold_results = pd.DataFrame(

    fold_results

)

display(

    table_fold_results

)

```

Code Snippet 5.2: Five-fold stratified cross-validation and Optuna hyperparameter optimization code

...	Fold	Original ROC-AUC	Engineered ROC-AUC	Difference
0	1	0.789916	0.792156	0.002240
1	2	0.788338	0.791330	0.002993
2	3	0.782034	0.781715	-0.000318
3	4	0.772368	0.774932	0.002565
4	5	0.797077	0.799986	0.002909

5.3 Evaluation Metrics

Model performance was assessed using the following classification metrics, which together give a more complete picture than accuracy alone on an imbalanced dataset:

- **Accuracy:** Proportion of total predictions that are correct.
- **Precision:** Proportion of predicted defaulters who are actual defaulters.
- **Recall:** Proportion of actual defaulters correctly identified by the model.
- **F1-Score:** Harmonic mean of precision and recall.
- **ROC-AUC:** Area under the Receiver Operating Characteristic curve, measuring the model's ability to rank defaulters above non-defaulters across all thresholds.
- **Confusion Matrix:** A breakdown of true positives, true negatives, false positives, and false negatives.

5.4 Results

5.4.1 Baseline Model Comparison (Five-Fold Cross-Validation)

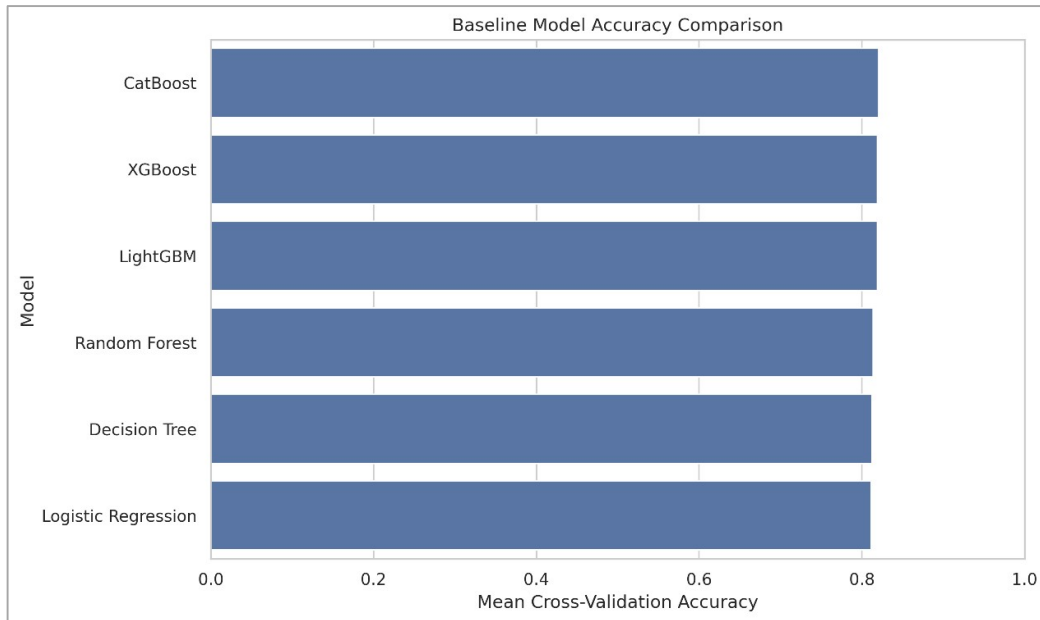
All six models were trained on the full 33-feature engineered representation and evaluated using five-fold stratified cross-validation. Table 5.1 reports the mean of each metric across folds.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
CatBoost	0.8209	0.6727	0.3716	0.4786	0.7880
XGBoost	0.8190	0.6617	0.3726	0.4766	0.7811
LightGBM	0.8189	0.6637	0.3684	0.4737	0.7806
Random Forest	0.8139	0.6346	0.3744	0.4709	0.7696
Decision Tree	0.8126	0.6365	0.3580	0.4577	0.7458
Logistic Regression	0.8117	0.6673	0.2971	0.4111	0.7650

Table 5.1: Baseline model comparison — five-fold stratified cross-validation (engineered feature set)

CatBoost is the top performer on every metric except precision, achieving the highest mean ROC-AUC (0.7880) and accuracy (82.09%). XGBoost and LightGBM form a closely clustered second tier, followed by Random Forest and Logistic Regression. The single Decision Tree is the weakest model by a clear margin, consistent with the high variance of an unpruned tree relative to ensemble methods.

Figure 5.1: Mean five-fold cross-validation accuracy by model (bar chart)



5.4.2 Final Optimized Model Performance

The top-performing baseline model, CatBoost, was tuned using Optuna-based Bayesian hyperparameter optimization and evaluated on the held-out test set (6,000 records) at the standard 0.5 decision threshold.

Model	Threshold	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Optimized CatBoost	0.50	0.8190	0.6709	0.3567	0.4658	0.7780

Table 5.2: Final Optuna-tuned CatBoost model — held-out test-set performance

The tuned model achieves 81.90% accuracy and a ROC-AUC of 0.7780 on the held-out split. This single-split test ROC-AUC is somewhat below the 0.7880 mean obtained via five-fold cross-validation (Table 5.1), illustrating the sampling variance between one particular held-out partition and a cross-validated estimate.

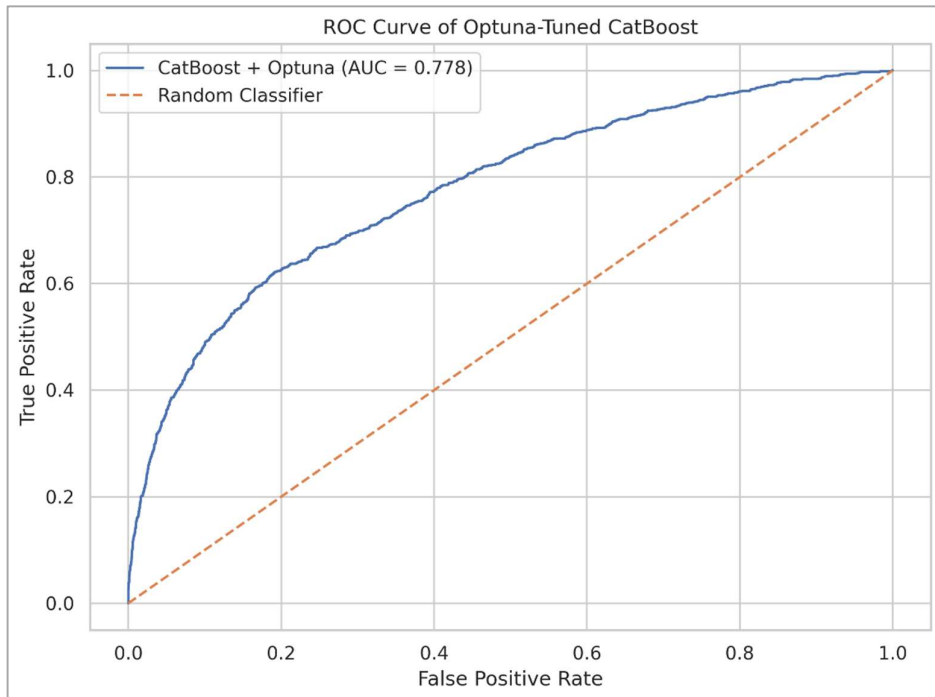


Figure 5.2: ROC curve of the Optuna-tuned CatBoost model on the held-out test set

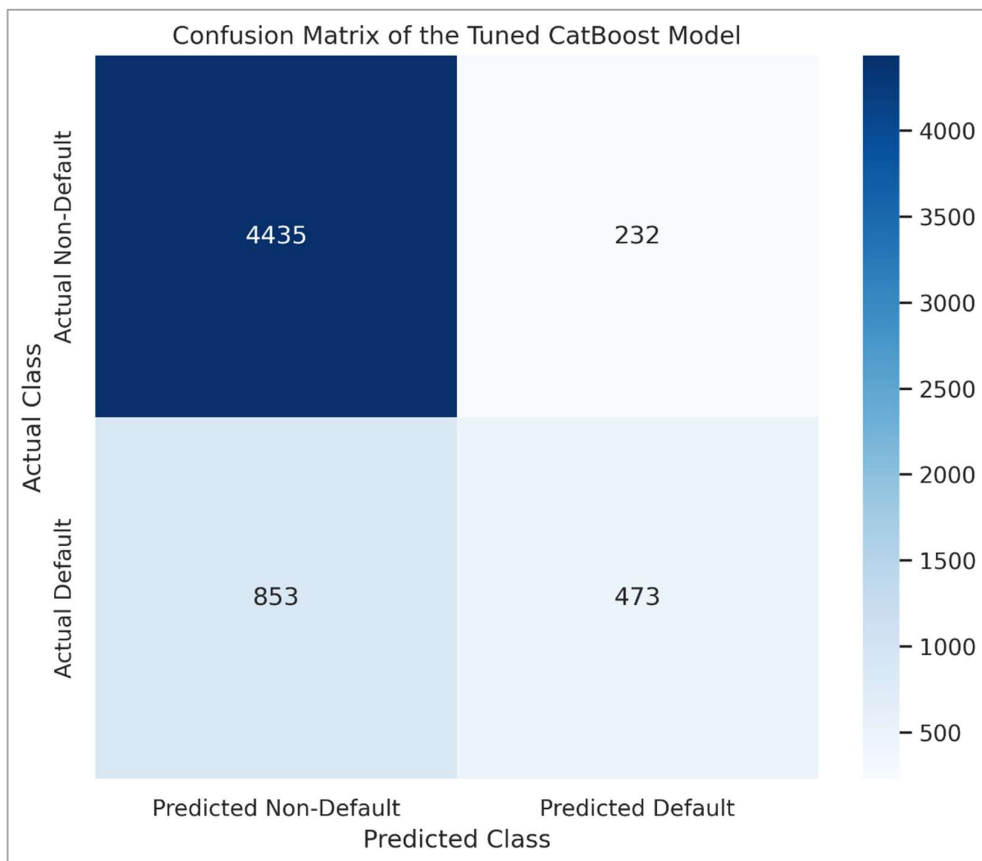


Figure 5.3: Confusion matrix of the tuned CatBoost model

5.4.3 Cross-Validation Robustness Check

To confirm that the ranking in Table 5.1 was not an artifact of a single train/test partition, Table 5.3 reports the mean and standard deviation of ROC-AUC for each model across the same five-fold stratified cross-validation.

Model	Mean ROC-AUC	Std ROC-AUC
CatBoost	0.7880	0.0087
XGBoost	0.7811	0.0079
LightGBM	0.7806	0.0096
Random Forest	0.7696	0.0066
Logistic Regression	0.7650	0.0090
Decision Tree	0.7458	0.0103

Table 5.3: Five-fold cross-validation ROC-AUC (mean $\pm$ std) across all six models

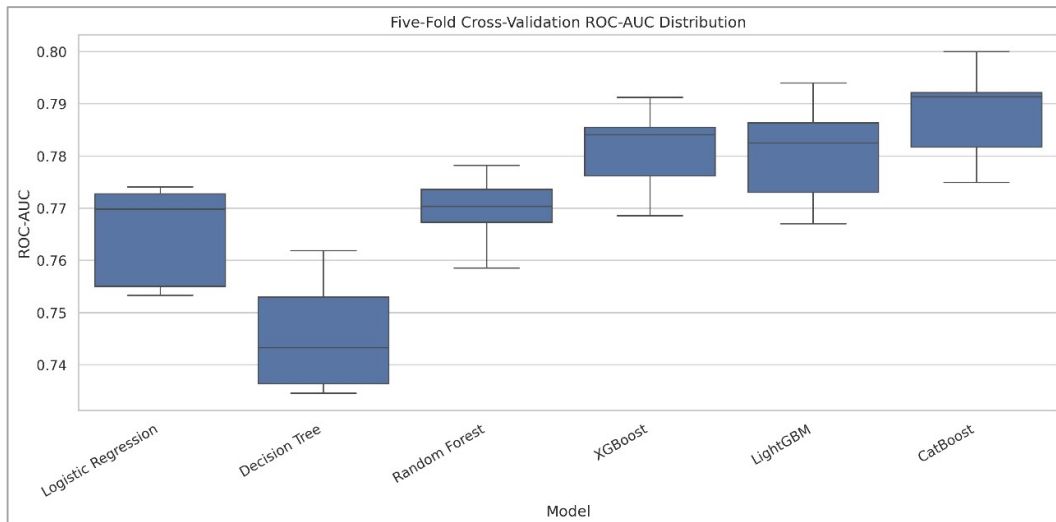


Figure 5.4: Cross-validation ROC-AUC distribution across models (box plot)

5.4.4 Ablation Study — Effect of Engineered Features

To isolate the effect of the ten engineered behavioral features, CatBoost was trained and evaluated on two feature configurations using an identical train/test split: the 23 original attributes only, and the original attributes plus the ten engineered features.

Feature Set	ROC-AUC	Accuracy	F1-Score	Avg. Precision
Original Features Only	0.7763	0.8188	0.4708	0.5554
Original + Engineered	0.7780	0.8190	0.4658	0.5652

Table 5.4: Ablation study — single held-out split (with vs. without engineered features)

On the single held-out split, adding the engineered features improved ROC-AUC by +0.0017 and Average Precision by +0.0098. To check robustness, the same comparison was repeated under five-fold stratified cross-validation.

Fold	Original ROC-AUC	Engineered ROC-AUC	Difference
1	0.7899	0.7922	+0.0022
2	0.7883	0.7913	+0.0030
3	0.7820	0.7817	-0.0003
4	0.7724	0.7749	+0.0026
5	0.7971	0.8000	+0.0029
Mean $\pm$ Std	0.7859 $\pm$ 0.0093	0.7880 $\pm$ 0.0098	+0.0021

Table 5.5: Five-fold cross-validation, original vs. engineered features (CatBoost)

The engineered feature set outperformed the original feature set in four of the five folds. A paired one-sided Wilcoxon signed-rank test on the fold-level ROC-AUC differences yielded a test statistic of 14.0 and $p = 0.0625$, which does not reach the conventional $\alpha = 0.05$ significance level at this sample size. This is a more conservative and honest conclusion than the single-split comparison alone would suggest: the engineered behavioral features provide a small, directionally consistent, but not statistically decisive improvement in ranking performance, since much of the raw PAY_*, BILL_AMT*, and PAY_AMT* data already implicitly encodes the behavioral signal that the engineered aggregates make explicit.

```

# -----
# Train CatBoost using original features only
# -----

ablation_original_model = CatBoostClassifier(
    iterations=300,
    learning_rate=0.05,
    depth=6,
    verbose=False,
    random_seed=RANDOM_STATE
)

ablation_original_model.fit(
    X_original_train,
    y_train
)

# Generate predictions.
original_predictions = (
    ablation_original_model
    .predict(X_original_test)
    .astype(int)
    .ravel()
)

# Generate probability predictions.
original_probabilities = (
    ablation_original_model
    .predict_proba(
        X_original_test
    )[:, 1]
)

```

```
# -----
# Create ablation results table
# -----

ablation_results = pd.DataFrame([

    {
        "Feature Set": "Original Features Only",

        "ROC-AUC": roc_auc_score(
            y_test,
            original_probabilities
        ),

        "Accuracy": accuracy_score(
            y_test,
            original_predictions
        ),

        "F1-Score": f1_score(
            y_test,
            original_predictions,
            zero_division=0
        ),

        "Average Precision": average_precision_score(
            y_test,
            original_probabilities
        )
    },

    {
        "Feature Set": "Original + Engineered Features",
```

```
        "ROC-AUC": roc_auc_score(
            y_test,
            engineered_probabilities
        ),

        "Accuracy": accuracy_score(
            y_test,
            engineered_predictions
        ),

        "F1-Score": f1_score(
            y_test,
            engineered_predictions,
            zero_division=0
        ),

        "Average Precision": average_precision_score(
            y_test,
            engineered_probabilities
        )
    }

])

display(ablation_results)
```

...	Feature Set	ROC-AUC	Accuracy	F1-Score	Average Precision
0	Original Features Only	0.776288	0.818789	0.47076	0.555418
1	Original + Engineered Features	0.777994	0.818955	0.46578	0.565227

Code Snippet 5.3: Ablation study and paired Wilcoxon signed-rank test code

5.4.5 Explainability — Feature Importance and SHAP Analysis

CatBoost's native feature-importance scores were computed for the tuned model on the full 33-feature representation. Table 5.6 reports the top eight predictors.

Rank	Feature	Importance (%)
1	PAY_0	18.83
2	CREDIT_UTILIZATION	7.43
3	MAX_PAY_STATUS	5.73
4	LIMIT_BAL	5.21
5	BILL_AMT1	5.18
6	SEVERE_DELAY_MONTHS	4.06
7	DELAYED_MONTHS	3.59
8	AVG_PAY_STATUS	3.50

Table 5.6: Top eight predictors by CatBoost feature importance

The most recent repayment status (PAY_0) remains by far the single strongest predictor (18.83% relative importance). Notably, five of the top eight predictors are engineered behavioral features — CREDIT_UTILIZATION, MAX_PAY_STATUS, SEVERE_DELAY_MONTHS, DELAYED_MONTHS, and AVG_PAY_STATUS — confirming that behavioral aggregation surfaces genuinely informative signal even though the raw repayment-status columns already carry much of it. SHAP (SHapley Additive exPlanations) analysis using a TreeExplainer was additionally performed on the held-out test set to provide both global feature-importance ranking and local, per-customer explanations.

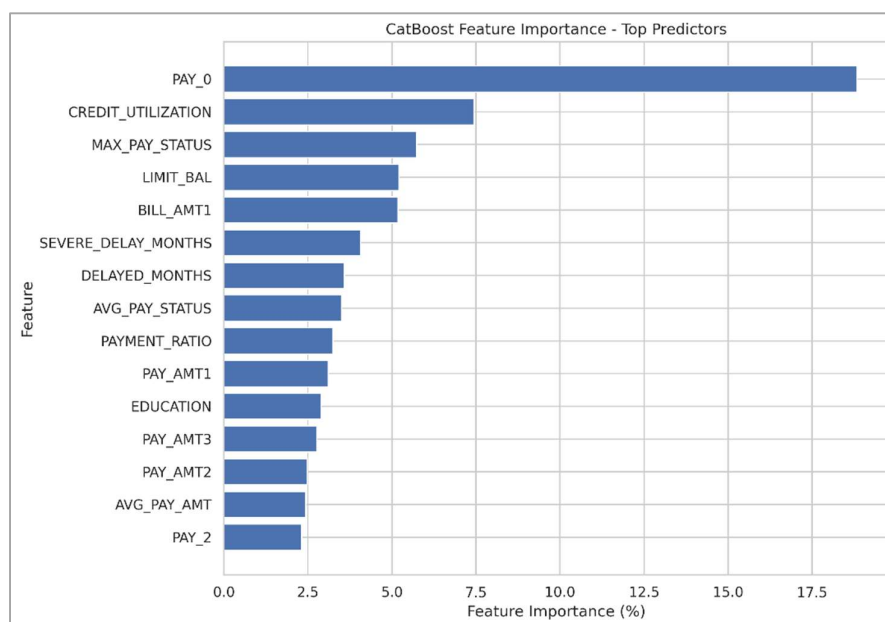


figure 5.5: CatBoost feature importance — top 15 predictors (engineered features highlighted)

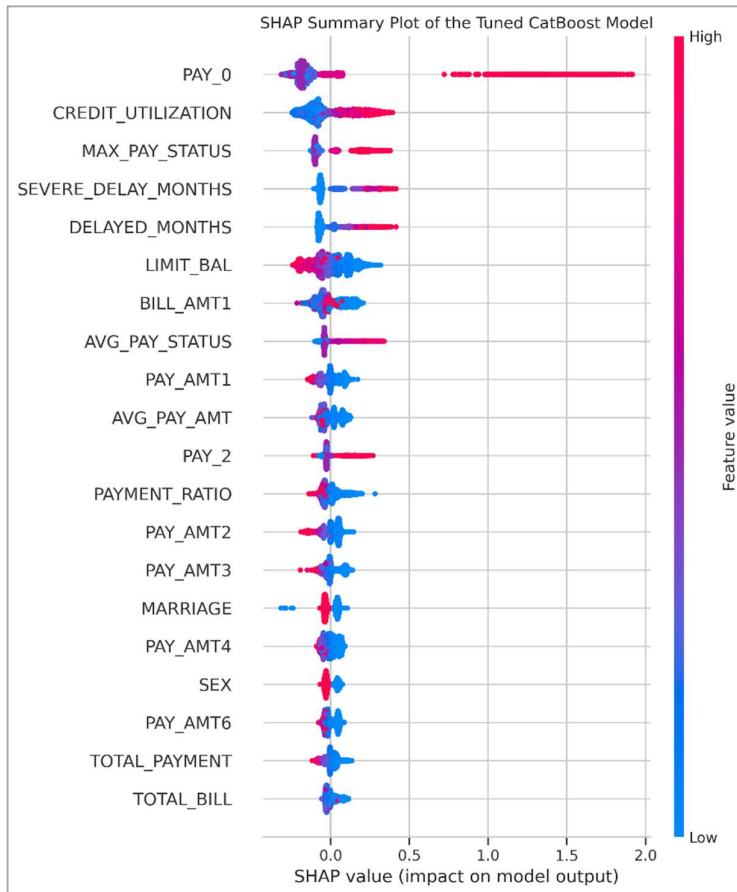


Figure 5.6: SHAP summary (beeswarm) plot showing feature impact and direction on model output

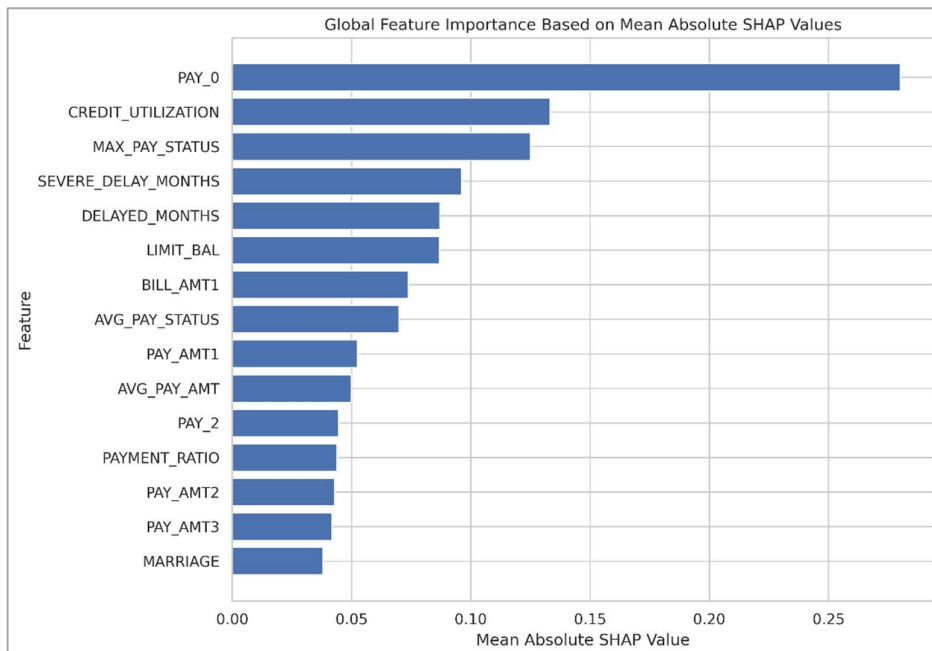


Figure 5.7: Mean absolute SHAP feature importance — top 15 predictors

SHAP ANALYSIS: CREATE EXPLAINER

```
# Create a TreeExplainer for the final optimized CatBoost model.
# TreeExplainer is designed for tree-based models such as CatBoost.
explainer = shap.TreeExplainer(
    optimized_catboost
)

print("SHAP TreeExplainer created successfully.")
```

SHAP TreeExplainer created successfully.

SHAP ANALYSIS: CALCULATE SHAP VALUES

```
# Calculate SHAP values for the held-out test set.
# We use X_test because these are unseen customer records
# used for final model evaluation.

shap_values = explainer.shap_values(
    X_test
)

print("SHAP values calculated successfully.")
print(
    f"SHAP matrix shape: {shap_values.shape}"
)
```

SHAP values calculated successfully.
SHAP matrix shape: (5993, 33)

Code Snippet 5.4: SHAP TreeExplainer setup and SHAP value computation code

5.5 Model Comparison Summary

Across every evaluation carried out — baseline five-fold cross-validation, the held-out test set, and the cross-validation robustness check — CatBoost consistently ranked first among the six models, with XGBoost and LightGBM forming a closely-matched second tier. The final tuned CatBoost model was therefore selected as the recommended model for credit card default prediction in this project, achieving a mean cross-validated ROC-AUC of 0.7880 and a held-out test-set ROC-AUC of 0.7780 at the standard 0.5 decision threshold.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This internship project developed and evaluated a complete machine learning pipeline for predicting credit card default using the UCI Credit Card Default dataset. Ten behavioral features — summarizing repayment consistency, delay frequency, credit utilization, and payment ratios — were engineered from the six-month repayment history and appended to the 23 original attributes, forming a 33-feature representation.

Among six supervised machine learning models compared using five-fold stratified cross-validation (Logistic Regression, Decision Tree, Random Forest, XGBoost, LightGBM, and CatBoost), CatBoost achieved the strongest performance (mean ROC-AUC = 0.7880 ± 0.0087 , accuracy = 82.09%). Optuna-based Bayesian hyperparameter tuning of CatBoost produced a final held-out test-set ROC-AUC of 0.7780.

A controlled ablation study showed that the engineered behavioral features provide a small, directionally consistent improvement in ROC-AUC on both a single held-out split (+0.0017) and under five-fold cross-validation (+0.0021, improving in four of five folds), though a paired one-sided Wilcoxon signed-rank test (statistic = 14.0, $p = 0.0625$) did not reach conventional statistical significance at this sample size.

Feature-importance analysis showed that five of the top eight predictors are engineered behavioral features, confirming that repayment-status magnitude and frequency, together with credit utilization, are the dominant drivers of the model's default predictions.

Overall, the project demonstrates how a structured data analytics pipeline — spanning data cleaning, exploratory analysis, behavioral feature engineering, systematic model comparison, rigorous statistical validation, and explainable AI — can be applied to a real-world financial risk problem, while highlighting the importance of cross-validation and significance testing rather than relying on single-split accuracy gains alone.

6.2 Future Scope

- **Larger-scale statistical validation:** Extend cross-validation to a larger number of folds or repeated resampling to obtain greater statistical power on the ablation question.
- **Class-imbalance handling:** Incorporate class-imbalance-aware training, such as cost-sensitive loss functions or resampling techniques, to improve the currently modest recall (0.36 – 0.37) for the minority default class.

- **Trajectory-based features:** Explore trajectory-based behavioral features, such as fitted slopes of utilization and delay over time, rather than simple aggregates and counts.
- **Fairness auditing:** Evaluate a fairness audit of model errors across demographic attributes (sex, education, marital status, age), given their relevance to responsible lending practice.
- **Real-time deployment:** Package the tuned CatBoost model behind a REST API or lightweight web application for real-time default-risk scoring.
- **Deep learning approaches:** Explore deep learning architectures (e.g., TabNet, feed-forward neural networks) as an additional comparison point against gradient boosting methods.
- **Larger datasets:** Validate the framework on larger and more recent financial datasets to improve scalability and generalizability beyond the Taiwan-based UCI dataset.

REFERENCES

- [1] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
- [2] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 3149–3157, 2017.
- [3] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [4] Y. C. Chang, K. H. Chang, and G. J. Wu, "Application of eXtreme gradient boosting trees in the construction of credit risk assessment models for financial institutions," *Applied Soft Computing*, vol. 73, pp. 914–920, 2018.
- [5] S. M. Lundberg and S. I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 4765–4774, 2017.
- [6] J. T. Hancock and T. M. Khoshgoftaar, "CatBoost for big data: an interdisciplinary review," *Journal of Big Data*, vol. 7, no. 94, 2020.
- [7] D. Björkegren and D. Grissen, "Behavior revealed in mobile phone usage predicts credit repayment," *World Bank Economic Review*, vol. 33, no. 3, pp. 534–547, 2019.
- [8] S. Bhatore, L. Mohan, and Y. R. Reddy, "Machine learning techniques for credit risk evaluation: a systematic literature review," *Journal of Banking and Financial Technology*, vol. 4, pp. 111–138, 2020.
- [9] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, pp. 2623–2631, 2019.
- [10] I. C. Yeh and C. H. Lien, "The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients," *Expert Systems with Applications*, vol. 36, no. 2, pp. 2473–2480, 2009.
- [11] Pedregosa, F. et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [12] UCI Machine Learning Repository, "Default of Credit Card Clients Data Set," <https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients> (accessed 2026).